



Process Synchronization Contd.

Dr. Jit Mukherjee



Synchronization Hardware

- Peterson's Solution -> software based
- Generalization - Any solution to critical section problem needs a lock
 - Race conditions are prevented by protecting critical section by lock
- Hardware features improve system efficiency. Critical Section -> Hardware Instructions
- Uniprocessor -> Disable the interrupt when shared variables are used
- Multiprocessor -> Disabling interrupt time consuming, message passing delay -> reduces system efficiency
- Hardware instruction either to test and modify content of a word or swap the content of two words.
- These are atomic instructions -> one uninterruptible unit

TestAndSet()

- TestAndSet() executed atomically
- If two TestAndSet() are executed concurrently in multi-core -> sequentially in some order.
- If machine supports TestAndSet() instruction -> a boolean variable Lock and initialize to false.

```
Boolean TestAndSet (boolean *target)
{
    Boolean rv = *target;
    *target = TRUE;
    return rv;
}

do
{
    while(TestAndSetLock(&lock))
        ; /*do nothing*/

    //critical section

    lock = FALSE;

    //remainder section
}while(TRUE)
```

Swap()

- Operates on two variables
- Executed atomically
- A boolean variable lock and initialized to false.
- Swap() and TestAndSet() satisfies mutual exclusion.
- Not bounded waiting

```
Void Swap (boolean *a, boolean *b)
{
    Boolean temp = *a;
    *a = *b;
    *b = temp;
}

do
{
    Key = TRUE;
    while (key == TRUE)
        Swap (&lock, &key);

    // critical section

    Lock = FALSE;

    // remainder section
}
```

TestAndSet() Bounded Waiting

```
Do{
    waiting [i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet (&lock);
    waiting[i] = FALSE;
    //critical section
    j= (i+1) %n;
    while((j!=i) && !waiting[j])
        J = (j+1) %n;
```

```
    if(j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;
        //remainder section
    }while (TRUE);
```

Semaphore

- Hardware based solution may become complicated for applications programmers
- A semaphore S is an integer, apart from initialization, is accessed by
 - Wait () (Termed as P)
 - Signal () (Termed as V)
- When one process modifies the semaphore value, other waits

```
Wait (S)
{
    While S <= 0
        ; // no operation
    S -- ;
}

Signal (S)
{
    S++;
}

Do //mutual exclusion
{
    Wait (mutex);

    // critical section

    Signal (mutex);

    //remainder section
}while (TRUE);
```

Semaphore Contd.

- Binary semaphore - [0, 1] -> mutex locks
- n processes shares a semaphore mutex, initialized to 1.
- Counting semaphore - can range over a domain
 - To control access a resource with finite number of instances
 - Semaphore initialized to the number of instances
 - One process wishes to use wait ()
 - Release -> signal ()
 - Semaphore = 0 -> all instances are used
- On other instances mutex can be used. P_1 with statement S_1 had be executed before P_2 with statement S_2
 - S_1; Signal (synch);
 - Wait (synch); S_2;

Spin Lock

- Busy waiting - when a process is in critical sections, others must loop continuously
- Busy waiting waste CPU cycle what can be used in other productive way.
- These semaphores are called Spin Lock
 - When wait time in lock is less, spin lock is useful
 - Problem multiprocessor system
- When a process executes Wait () and finds semaphore value is not positive, the process can block itself
 - Waiting queue associated with the semaphore
- After other process gives Signal (), the waiting process performs a Wakeup() call.
 - Waiting state to ready state.

Spin Lock

- Classical implementation of mutex never goes negative, but here it can
 - Number of process in waiting state
- FIFO queue
- Wait () and Signal () at same time
 - Uniprocessor -> remove interrupt
- In SMP better to use Spin Lock.
- When critical section is long -> busy waiting extremely inefficient
- This may initiate a deadlock

```
Typedef struct
{
    int value;
    Struct process *list;
}semaphore;

Wait (semaphore *S)
{
    S-> value -;
    if (S->value < 0)
    {
        Add the process S->list;
        block();
    }
}

Signal (semaphore *S)
{
    S->value++;
    if (S->value <= 0)
    {
        Remove process P S->list;
        Wakeup (P)
    }
}
```

The Bounded Buffer Problem

- Producer Consumer Problem
- N sized bounded buffer. Initialization - mutex $\rightarrow$ 1, empty $\rightarrow$ N, full $\rightarrow$ 0

```
do
{
    // produce an item in nextp
    Wait(empty);
    Wait(mutex);

    //add nextp to buffer

    Signal (mutex);
    Signal (full);
}while (TRUE);
```

```
do
{
    Wait (full);
    Wait (mutex);

    // remove an item from buffer to next

    Signal (mutex);
    Signal (empty);

    // consume the item in nextc
}while (TRUE);
```